ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic – Machine Learning from Disaster (uploaded train.csv included in this project)

Objective

Train and save one machine-learning classification pipeline, demonstrate online, batch, and streaming inference, and serve predictions through FastAPI.

1. Online Inference vs Batch Inference

Online inference predicts one individual record when a request arrives and returns a result immediately. Batch inference processes a group of records together. Online inference is useful for individual, low-latency requests, while batch inference is efficient for many records.

2. Streaming Inference vs Batch Inference

Streaming inference processes records as they arrive, often one at a time, so results can be produced with low delay. Batch inference collects a group of records and processes them together. This assignment simulates streaming by processing 10 held-out records sequentially with a one-second delay.

3. Why FastAPI is Used for Model Serving

FastAPI exposes a trained model through HTTP endpoints so other applications can request predictions. Pydantic validates incoming data, Swagger UI at /docs enables interactive testing, and the API returns structured JSON responses.

4. Why Reuse the Same Trained Model?

Reusing the same saved pipeline keeps preprocessing and model behavior consistent across online, batch, streaming, and API inference. It avoids discrepancies caused by retraining or changing preprocessing and improves reproducibility.

Implementation and Results

The project uses a Logistic Regression classifier inside a scikit-learn Pipeline. Numeric missing values are median-imputed and scaled; categorical missing values are imputed with the most frequent value and one-hot encoded. The labelled data is split into training and test portions with stratification and random_state=42. Inference demonstrations use held-out test records without their labels.

Measured test accuracy: 80.45% (computed by train_model.py on the uploaded dataset).

Evidence to capture: online prediction, 20-row batch predictions, 10 streaming predictions, successful /health and /predict requests, invalid /predict validation error (HTTP 422), and Python client response.

Conclusion

The project demonstrates training and saving one pipeline, using it for three inference patterns, and serving predictions through a REST API. All numeric results should correspond to actual execution outputs.